Reviewer Pack — Arithmetic Hierarchy Complexity of Boolean Functions and Cir…

Entry 827731e68ead · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 05:42:44 UTC

Arithmetic Hierarchy Complexity of Boolean Functions and Circuit Monotone Width

Entry ID: `827731e68ead` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-04 05:42:44 UTC

1. Conjecture

Field A (mathematical branch): Arithmetic Hierarchy Theory **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

> For any given boolean function f: {0,1}^n -> {0,1}, the arithmetic hierarchy complexity AH(f) of f is linearly correlated with its circuit monotone width w_mon(f), such that AH(f) = Θ(w_mon(f)).

Rationale (proposer's reasoning):

> Arithmetic Hierarchy Theory provides a framework to describe computational problems by their difficulty level in terms of Turing machines. By mapping boolean functions to arithmetic hierarchies, we can potentially uncover structural properties related to circuit complexity measures like monotone width. This bridge might expose deep connections between abstract algebraic structures and the concrete complexity of computing functions.

Taxonomy category: `ArithmeticHierarchy` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `7fa5160f7ce3646c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for a minimum of 100 randomly generated boolean functions f: {0,1}^n -> {0,1}, the correlation coefficient between AH(f) and w_mon(f) exceeds 0.9, with p-value ≤ 0.01.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"arithmetic hierarchy theory" AND "boolean function complexity"` - `"circuit monotone width" IN TITLE AND "arithmetic hierarchy"` - `"Boolean circuit complexity" AND ".monotone width." AND arithmetic`

Top relevant hits considered: - [http://arxiv.org/abs/2503.14756v3] SceneEval: Evaluating Semantic Coherence in Text-Conditioned 3D Indoor Scene Synthesis - [http://arxiv.org/abs/2408.02211v2] SceneMotifCoder: Example-driven Visual Program Learning for Generating 3D Object Arrangements - [http://arxiv.org/abs/2509.13291v1] Towards an Embodied Composition Framework for Organizing Immersive Computational Notebooks - [s2:10.4230/LIPIcs.FSTTCS.2022.12] Robustly Separating the Arithmetic Monotone Hierarchy Via Graph Inner-Product - [s2:8477f43ea5ed4853c56bae6f9a0a950 Complexity Theoretic aspects of Rank Rigidity and circuit evaluation[HBNI Th 8]

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def arithmetic_hierarchy_complexity(f):
        # Placeholder function. This is a stub and should be replaced with actual computation.
        return len(f) / 2

    def circuit_monotone_width(f):
        # Placeholder function. This is a stub and should be replaced with actual computation.
        return sum(1 for bit in f if bit == 1)

    n_values = [30, 35, 40]
    results = []

    for n in n_values:
        for _ in range(10):  # Ensure at least 30 instances per seed
            f = generate_random_boolean_function(n)
            ah_f = arithmetic_hierarchy_complexity(f)
            w_mon_f = circuit_monotone_width(f)
```

```
            results.append((ah_f, w_mon_f))

    if not results:
        return {
            "metric_name": "correlation_coefficient",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    ah_values = [ah for ah, _ in results]
    w_mon_values = [w_mon for _, w_mon in results]

    mean_ah = sum(ah_values) / len(ah_values)
    mean_w_mon = sum(w_mon_values) / len(w_mon_values)

    covariance = sum((ah - mean_ah) * (w_mon - mean_w_mon) for ah, w_mon in results) / len(results
    variance_ah = sum((ah - mean_ah)**2 for ah in ah_values) / len(ah_values)
    variance_w_mon = sum((w_mon - mean_w_mon)**2 for w_mon in w_mon_values) / len(w_mon_values)

    correlation_coefficient = covariance / (math.sqrt(variance_ah) * math.sqrt(variance_w_mon))

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": abs(correlation_coefficient) > 0.9,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]  # Default to fi

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value)**2 for result in
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means the pre-registered support condition was not unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14491
2	preregistration	ollama_remote	glm4:latest	0	0	17454
3	novelty	ollama_remote	glm4:latest	0	0	11368
4	novelty	ollama_remote	glm4:latest	0	0	27036
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16294
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	38334
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	39760
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14898
9	judge	ollama_remote	glm4:latest	0	0	33693

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 213328 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/827731e68ead.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/827731e68ead.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/827731e68ead.tar.gz` (if generated)